

Experiment 03

LIBRARY MANAGEMENT SYSTEM

Title: Library Management System using OOP Concepts

Objective / Aim

To design and implement a Library Management System in Java using classes and objects, demonstrating encapsulation, constructors, getters and setters, inheritance, method overloading, method overriding, abstraction, and interfaces.

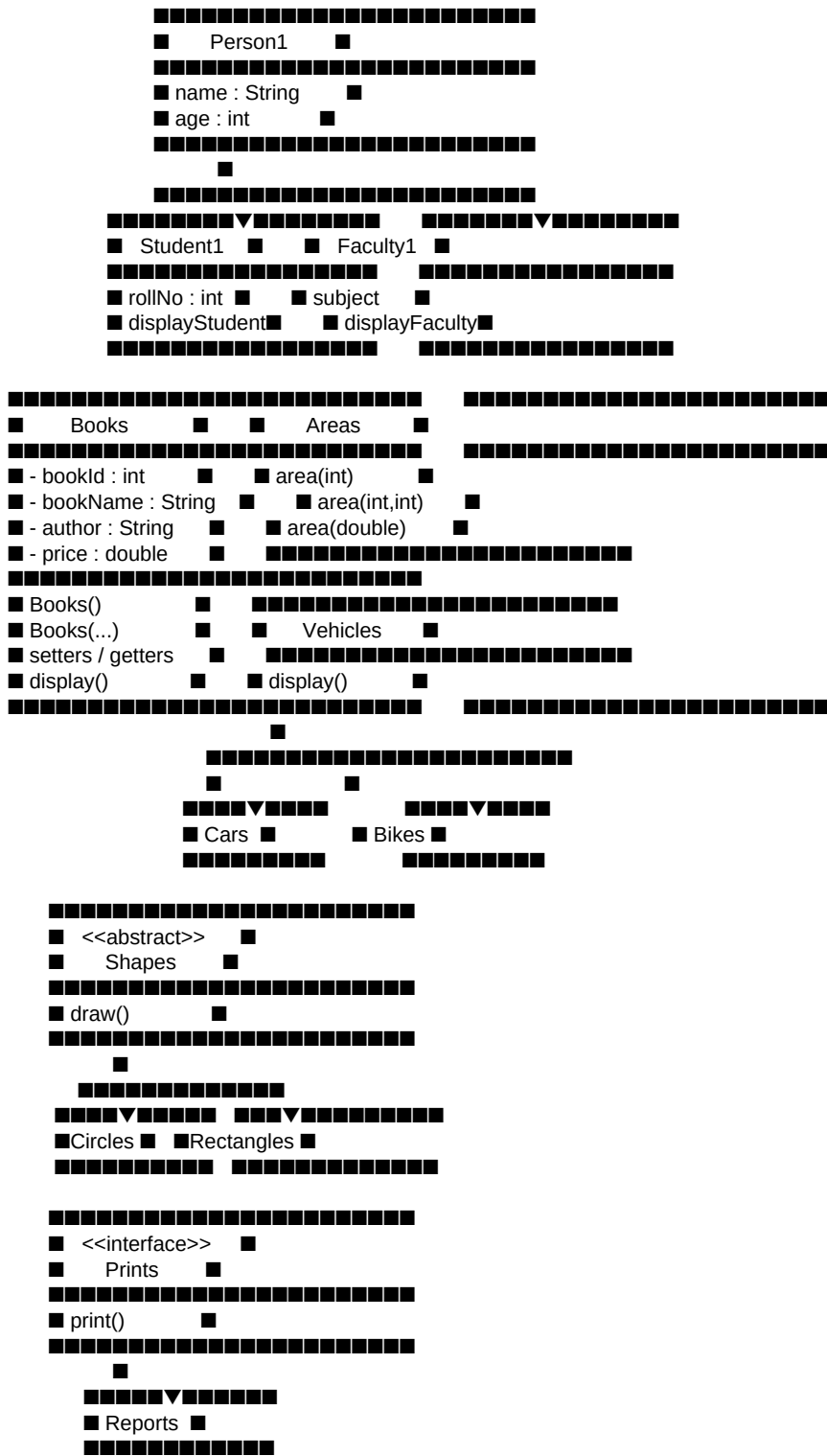
System Requirements

Hardware	Software
Standard PC/Laptop	Windows / Linux / macOS
Intel Core i3/i5 or equivalent	JDK 17 or JDK 21
Minimum 4 GB RAM	Java compiler (javac) and runtime (java)
Minimum 500 MB free storage	Eclipse / IntelliJ IDEA / NetBeans / VS Code

Algorithm / Logical Description

1. Create the Books class with private bookId, bookName, author, and price.
2. Provide default and parameterized constructors to initialize book objects.
3. Implement setter methods to assign values and getter methods to access values.
4. Implement display() to show book details.
5. Create Person1 as a parent class with common name and age properties.
6. Create Student1 and Faculty1 by inheriting Person1 and display their details.
7. Create Areas with overloaded area() methods for square, rectangle, and circle.
8. Create Vehicles and override display() in Cars and Bikes.
9. Create abstract class Shapes and implement draw() in Circles and Rectangles.
10. Create Prints interface and implement it in Reports.
11. Create objects in main() and demonstrate all the above OOP concepts.
12. Compile and execute the program and observe the output.

UML Class Diagram



Source Code

```
// Class Books demonstrates Encapsulation,
// Constructors, Getters and Setters
class Books {

    private int bookId;
    private String bookName;
    private String author;
    private double price;

    // Default Constructor
    public Books() {
        System.out.println("Default Constructor Called");
    }

    // Parameterized Constructor
    public Books(int id, String name, String author, double price) {
        this.bookId = id;
        this.bookName = name;
        this.author = author;
        this.price = price;
    }

    // Setter methods
    public void setBookId(int id) { bookId = id; }
    public void setBookName(String name) { bookName = name; }
    public void setAuthor(String a) { author = a; }
    public void setPrice(double p) { price = p; }

    // Getter methods
    public int getBookId() { return bookId; }
    public String getBookName() { return bookName; }
    public String getAuthor() { return author; }
    public double getPrice() { return price; }

    void display() {
        System.out.println("Book ID : " + bookId);
        System.out.println("Book Name : " + bookName);
        System.out.println("Author : " + author);
        System.out.println("Price : " + price);
    }
}

// Parent class - Inheritance
class Person1 {
    String name = "priya";
    int age = 22;
}

class Student1 extends Person1 {
    int rollNo = 156;

    void displayStudent() {
        System.out.println("\nStudent Details");
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
        System.out.println("Roll No : " + rollNo);
    }
}

class Faculty1 extends Person1 {
    String subject = "Java";

    void displayFaculty() {
        System.out.println("\nFaculty Details");
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
        System.out.println("Subject : " + subject);
    }
}

// Method Overloading
class Areas {
```

```

void area(int side) {
    System.out.println("\nArea of Square = " + (side * side));
}

void area(int length, int breadth) {
    System.out.println("Area of Rectangle = " + (length * breadth));
}

void area(double radius) {
    System.out.println("Area of Circle = " + (3.14 * radius * radius));
}
}

// Method Overriding
class Vehicles {
    void display() {
        System.out.println("\nThis is a Vehicle");
    }
}

class Cars extends Vehicles {
    void display() {
        System.out.println("This is a Car");
    }
}

class Bikes extends Vehicles {
    void display() {
        System.out.println("This is a Bike");
    }
}

// Abstract class
abstract class Shapes {
    abstract void draw();
}

class Circles extends Shapes {
    void draw() {
        System.out.println("\nDrawing Circle");
    }
}

class Rectangles extends Shapes {
    void draw() {
        System.out.println("Drawing Rectangle");
    }
}

// Interface
interface Prints {
    void print();
}

class Reports implements Prints {
    public void print() {
        System.out.println("\nPrinting Library Report");
    }
}

// Main class
public class Library {
    public static void main(String[] args) {

        Books b1 = new Books();
        b1.setBookId(167);
        b1.setBookName("Java Programming");
        b1.setAuthor("James Gosling");
        b1.setPrice(599);

        System.out.println("\nBook Details");
        b1.display();

        Books b2 = new Books(102, "Python", "Guido", 700);
    }
}

```

```
System.out.println("\nParameterized Constructor");  
b2.display();
```

```
Student1 s = new Student1();  
s.displayStudent();
```

```
Faculty1 f = new Faculty1();  
f.displayFaculty();
```

```
Areas a = new Areas();  
a.area(6);  
a.area(10, 24);  
a.area(3.7);
```

```
Vehicles v;  
v = new Cars();  
v.display();
```

```
v = new Bikes();  
v.display();
```

```
Shapes sh;  
sh = new Circles();  
sh.draw();
```

```
sh = new Rectangles();  
sh.draw();
```

```
Reports r = new Reports();  
r.print();
```

```
    }  
}
```

Output

Default Constructor Called

Book Details

Book ID : 167

Book Name : Java Programming

Author : James Gosling

Price : 599.0

Parameterized Constructor

Book ID : 102

Book Name : Python

Author : Guido

Price : 700.0

Student Details

Name : priya

Age : 22

Roll No : 156

Faculty Details

Name : priya

Age : 22

Subject : Java

Area of Square = 36

Area of Rectangle = 240

Area of Circle = 43.0136

This is a Car

This is a Bike

Drawing Circle

Drawing Rectangle

Printing Library Report

Completion and Execution Commands

1. Save the program as **Library.java**.
2. Open Command Prompt / Terminal and move to the folder containing the file.
3. Compile: **javac Library.java**
4. Execute: **java Library**
5. If there are no compilation errors, the program displays the library and OOP demonstrations.

Result

The Library Management System was successfully implemented and executed in Java. The program demonstrates encapsulation, constructors, getters and setters, inheritance, method overloading, method overriding, abstraction, and interface implementation.